1장 사용자 수에 따른 규모확장성

요청 처리 과정

클라이언트 —요청→ DNS (제3사업자가 제공하는 웹 서비스)
← IP address

IP address로 변환된 요청 → 웹 서버

응답 → JSON 이 HTML

수직적 규모 확장 vs 수평적 규모 확장

수직적 규모 확장
- 서버에 고사양 자원을 추가하여 성능 개선
  - ↳ 한계 : 한 대의 서버에 CPU나 메모리를 무한대 추가 만듦 + 서버 장애 발생 시 대처 불가
  - ↓ 다중화나 자동복구가 없음.

수평적 규모 확장
- 더 많은 서버를 추가해 개선
- 그래서 수평적 규모확장이 더 중요.

※ 데이터 센터만 백엔드 배포파일 + DB + α 등등 통째로 여러 벌 두는 일

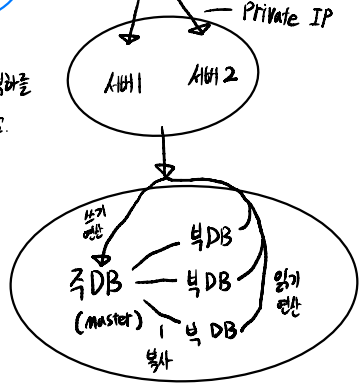
캐시 / CDN

캐시 : 자주 접근하는데 갈아끼우지 않는 데이터

CDN : 정적 데이터 전달.

로드 밸런서

웹서버에 트래픽 부하를 고르게 분산하는 역할.



DB 다중화

TIP : 애플리케이션의 성능은 데이터베이스를 얼마나 자주 호출하는지에 좌우됨.

데이터 베이스

RDB ⇒ 조인 지원함.
- MySQL
- 오라클DB
- PostgreSQL

Non-RDB ⇒ 조인 지원 안함.

⇒ 대부분 DB는 RDB를 사용하지만,
- 아주 낮은 응답지연시간 요구
- 비정형 데이터를 다룸
- 데이터를 직렬화 하거나 역직렬화 만든 양의 데이터를 저장만해도 됨

⇒ 이럴 땐 비관계형 DB가 쓰이긴 한다.

⇒ 수직적 확장 잘 사용안함. 단점이 너무 많음.

⇒ 데이터 베이스 확장 → 한개의 DB를 샤드라고 하여 해시함수를 사용해서 각 DB에 저장한다.

↳ 부하를 줄이기 위해서 관계형 아닌 NoSQL로 사용.

⇒ 이건 샤드 확장 법

전에 배운 복제 확장 법이랑은 다른 방법.

Master ⟷ Slave
- 쓰기
- 읽기

메세지 큐

↳ 비동기 통신 컴포넌트

생산자 → 메세지큐 → 소비자

↳ 일단 사용해보고 큐가 자주 비면 동기로 전환

↳ 비동기 큰 데이터를 위주로 사용

도구

로그 : 시스템에 일어난 구체적인 사건 → 행동로그, 에러로그 등...

메트릭 : 하드웨어 사용량, 로그들의 통계
- ↓ 호스트단위 메트릭.
- ↓ 핵심 비즈니스 메트릭.
- → 종합 메트릭.

⇒ 자동화, 오케스트레이션

⇓ 각 서비스마다 수평적 가능하지만 도구 컴포넌트를 따로 만들어서 사용

2장 개략적인 규모 추정

용량 KB MB GB TB PB

응답지연 ns μs ms
⇓
0.001초

가용성 ⇒ 퍼센트로 표현하여 100%는 서비스 도중 한번도 시스템이 중단된 적이 없는 뜻.